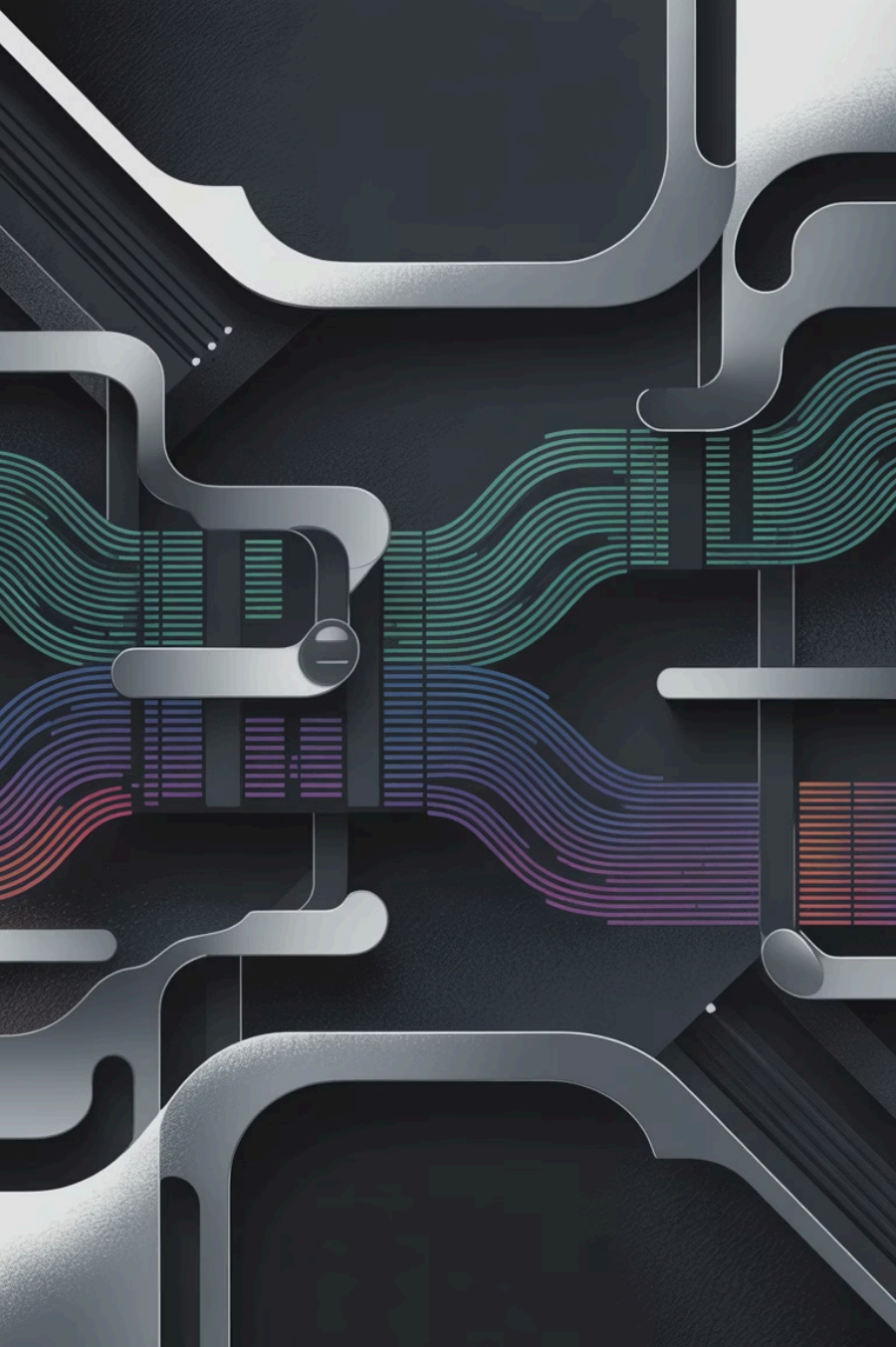




Module 14- Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence

Today's Learning Objectives

01

Understanding Few-Shot Prompting

Learn how to leverage examples to guide LLM behavior and improve response quality without fine-tuning.

03

Formatting Best Practices

Develop systematic approaches to structuring prompts with clear delimiters and consistent formatting.

02

Exemplar Design Principles

Master the art of crafting consistent, high-quality examples that establish clear patterns for the model.

04

Production Implementation

Apply prompting patterns to real-world LLM pipelines with practical code examples and validation strategies.

The Challenge: Getting Consistent LLM Outputs

When working with large language models in production, one of the biggest challenges is achieving consistent, reliable outputs. Without proper guidance, LLMs can produce responses that vary wildly in format, style, and quality—even when given similar inputs.

This inconsistency creates downstream problems in data pipelines. Parsing failures, validation errors, and unpredictable behavior make it difficult to build robust systems that depend on LLM-generated content.

Few-shot prompting offers a solution by teaching the model through examples rather than extensive fine-tuning or lengthy instructions.



What is Few-Shot Prompting?

Few-shot prompting is a technique where you provide the LLM with a small number of example input-output pairs before asking it to process a new input. These examples act as a mini-training set that demonstrates the desired behavior.

The model learns patterns from these exemplars and applies them to new, unseen inputs. This approach is particularly powerful because it requires no model retraining and can be adjusted instantly by changing the examples.

Key advantages:

- No fine-tuning or model updates required
- Fast iteration and experimentation
- Transparent and easily debuggable
- Works across different model providers



Quick Definition

Zero-shot: No examples provided

One-shot: Single example given

Few-shot: Multiple examples (typically 2-5)

Many-shot: Dozens or hundreds of examples

The Spectrum of Prompting Approaches

1

Zero-Shot

Direct instruction without examples. Fast but often inconsistent for complex tasks.

2

Few-Shot

2-5 carefully crafted examples. Optimal balance of token efficiency and guidance.

3

Many-Shot

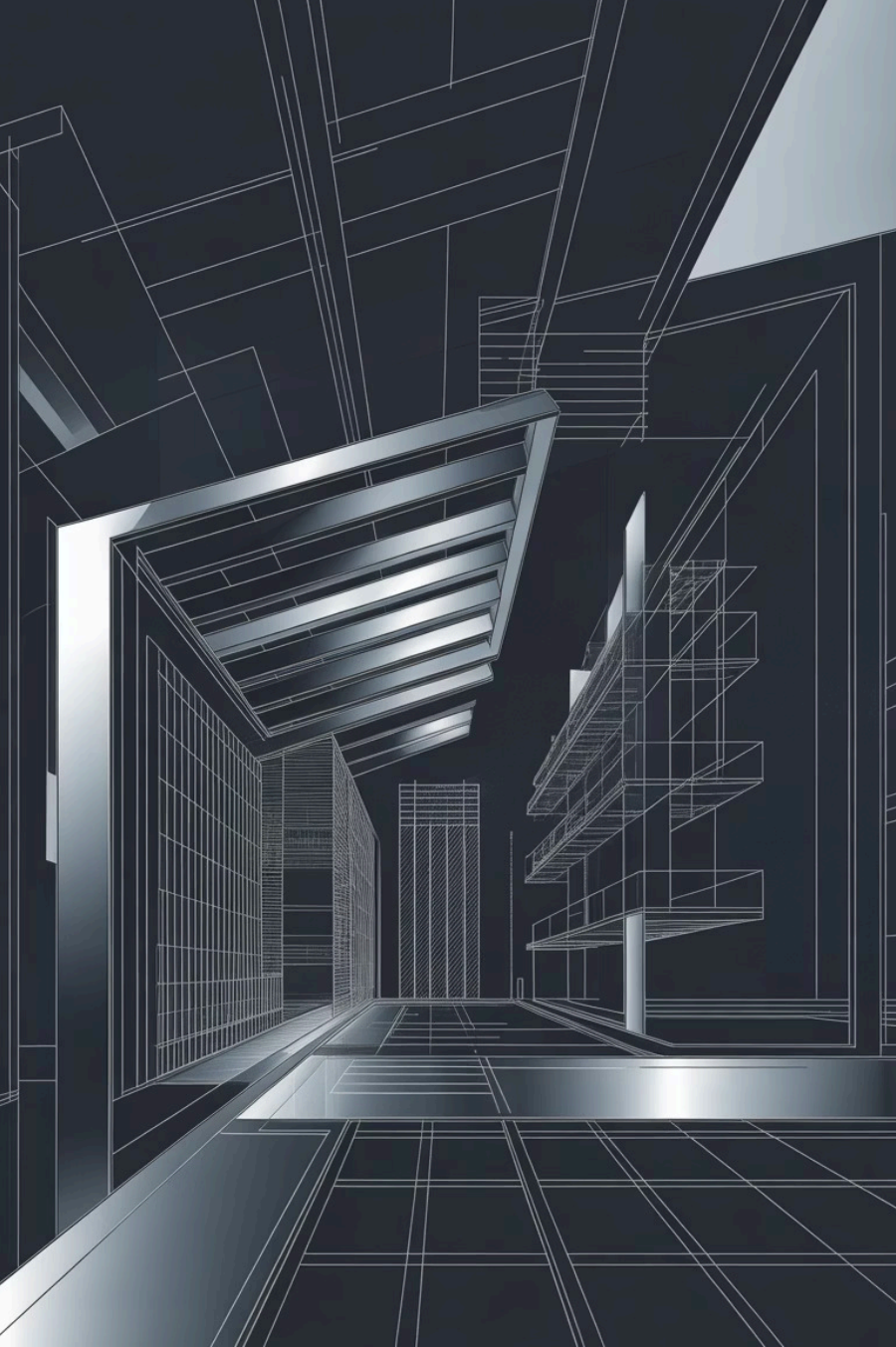
10+ examples for complex patterns. Higher token cost but maximum consistency.

4

Fine-Tuning

Custom model training on thousands of examples. Highest setup cost and effort.

For most production LLM pipelines, few-shot prompting provides the sweet spot between performance and practicality. It gives you enough control to ensure consistent outputs without the overhead of fine-tuning.



Anatomy of a Few-Shot Prompt

1

System Message

Sets the overall context, role, and behavior guidelines for the model.

2

Task Description

Clear explanation of what the model should do, including output format requirements.

3

Exemplar Set

2-5 high-quality input-output pairs demonstrating the desired pattern.

4

New Input

The actual data you want the model to process, formatted consistently with exemplars.

Example: Named Entity Recognition in Research Papers

System: You are an NER system that extracts entities from academic papers.

Task: Extract all mentions of models, datasets, and metrics from the text.

Return JSON with three lists: models, datasets, metrics.

Example 1:

Input: "We trained BERT on the SQuAD dataset and achieved 92% F1 score."

Output: {"models": ["BERT"], "datasets": ["SQuAD"], "metrics": ["F1 score"]}

Example 2:

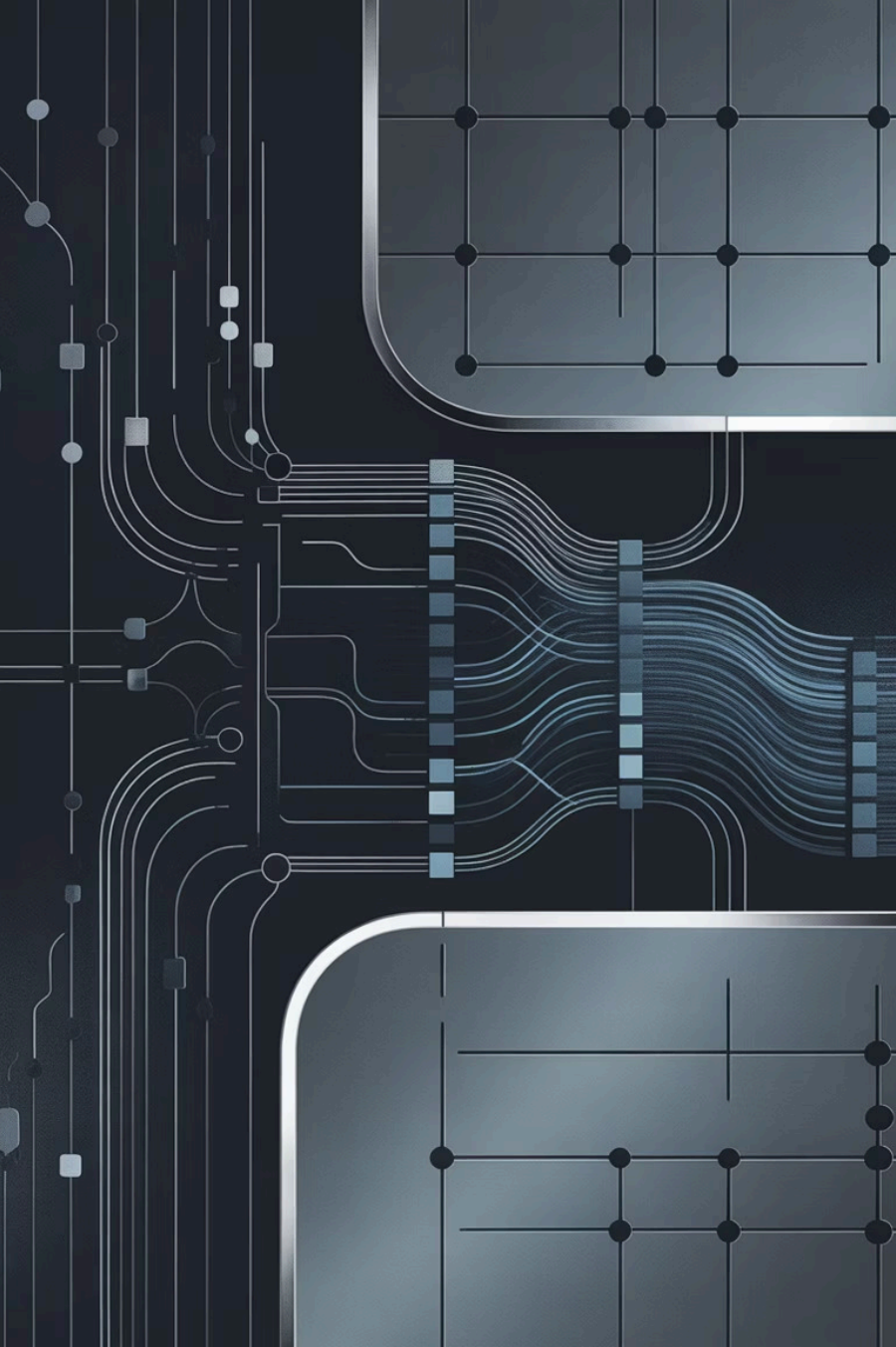
Input: "GPT-3 was evaluated on GLUE and SuperGLUE benchmarks."

Output: {"models": ["GPT-3"], "datasets": ["GLUE", "SuperGLUE"], "metrics": []}

Now process this text:

"Our transformer model fine-tuned on WikiText-103 achieved perplexity of 18.2."

Notice how the examples establish a clear pattern: consistent JSON structure, proper entity categorization, and handling of edge cases like empty lists.

An abstract graphic on the left side of the slide, featuring a dark background with glowing blue and white lines representing circuitry or data flow. There are several rectangular panels with grid patterns and dots, suggesting a technical or computational theme.

Why Few-Shot Prompting Works

Pattern Recognition at Scale

Large language models are trained on trillions of tokens and excel at recognizing patterns. When you provide structured examples, the model identifies the underlying pattern and applies it to new inputs.

The model's attention mechanism focuses on the relationships between your examples and the new input, effectively performing in-context learning.

Implicit Task Specification

Examples communicate requirements more precisely than instructions alone. They show edge cases, formatting details, and implicit rules that would take many words to explain.

This is especially valuable for complex tasks like data extraction, transformation, or classification where subtle formatting matters.

The Power of Consistency: Why Exemplar Quality Matters

Format Consistency

All examples must follow identical structure. Inconsistent formatting confuses the model and leads to unpredictable outputs.

Difficulty Progression

Start with simple cases, then show progressively more complex examples that cover edge cases.

Representative Coverage

Examples should span the diversity of inputs you expect, including common patterns and important exceptions.

Formatting Exemplars: Delimiter Strategy

Clear delimiters are crucial for helping the model understand where inputs end and outputs begin. Consistent delimiter usage prevents the model from confusing example data with the actual task.

Common Delimiter Patterns

- `### Input:` and `### Output:`
- Triple backticks for code: `````
- XML-style tags: `<input>` and `<output>`
- Markdown headers with clear hierarchy
- JSON structure with explicit keys

Best Practices

- Use the same delimiters across all examples
- Make delimiters visually distinct from content
- Avoid delimiters that appear in your data
- Consider using semantic markers like "Q:" and "A:"

Practical Example: Text Classification for Research Papers

Classify research papers into categories: NLP, Computer Vision, Robotics, or Other.

Example 1:

Title: "Attention Is All You Need"

Abstract: "We propose the Transformer, based solely on attention mechanisms..."

Category: NLP

Example 2:

Title: "ImageNet Classification with Deep Convolutional Neural Networks"

Abstract: "We trained a large CNN on ImageNet dataset..."

Category: Computer Vision

Example 3:

Title: "Mastering Chess through Self-Play"

Abstract: "We present a reinforcement learning approach for game AI..."

Category: Other

Now classify:

Title: "BERT: Pre-training of Deep Bidirectional Transformers"

Abstract: "We introduce BERT, designed to pre-train deep representations..."

Building Your Exemplar Set: A Systematic Approach



Collect Representative Data

Gather diverse examples from your actual dataset that cover common patterns and important edge cases.



Manually Label High-Quality Examples

Create gold-standard outputs for each example. This is your source of truth for model behavior.



Test and Validate

Run your prompt on held-out data to measure accuracy. Iterate on examples that produce errors.



Optimize for Token Efficiency

Reduce example length while maintaining clarity. Every token costs money and latency.



How Many Examples Do You Need?

The optimal number of examples depends on task complexity and desired consistency. Research and practical experience suggest clear patterns.



Simple Tasks (1-2 examples)

Format conversion, basic classification with obvious categories, or simple extraction tasks work well with minimal examples.



Moderate Tasks (3-5 examples)

Most production use cases fall here: entity extraction, text transformation, structured data generation, or multi-class classification.



Complex Tasks (6-10 examples)

Tasks requiring nuanced judgment, handling many edge cases, or producing complex structured outputs benefit from more examples.

Beyond 10 examples, you hit diminishing returns. At that point, consider fine-tuning or using retrieval-augmented generation.

Token Economics: Balancing Quality and Cost

The Token Trade-off

Every example in your prompt consumes input tokens, which directly impacts:

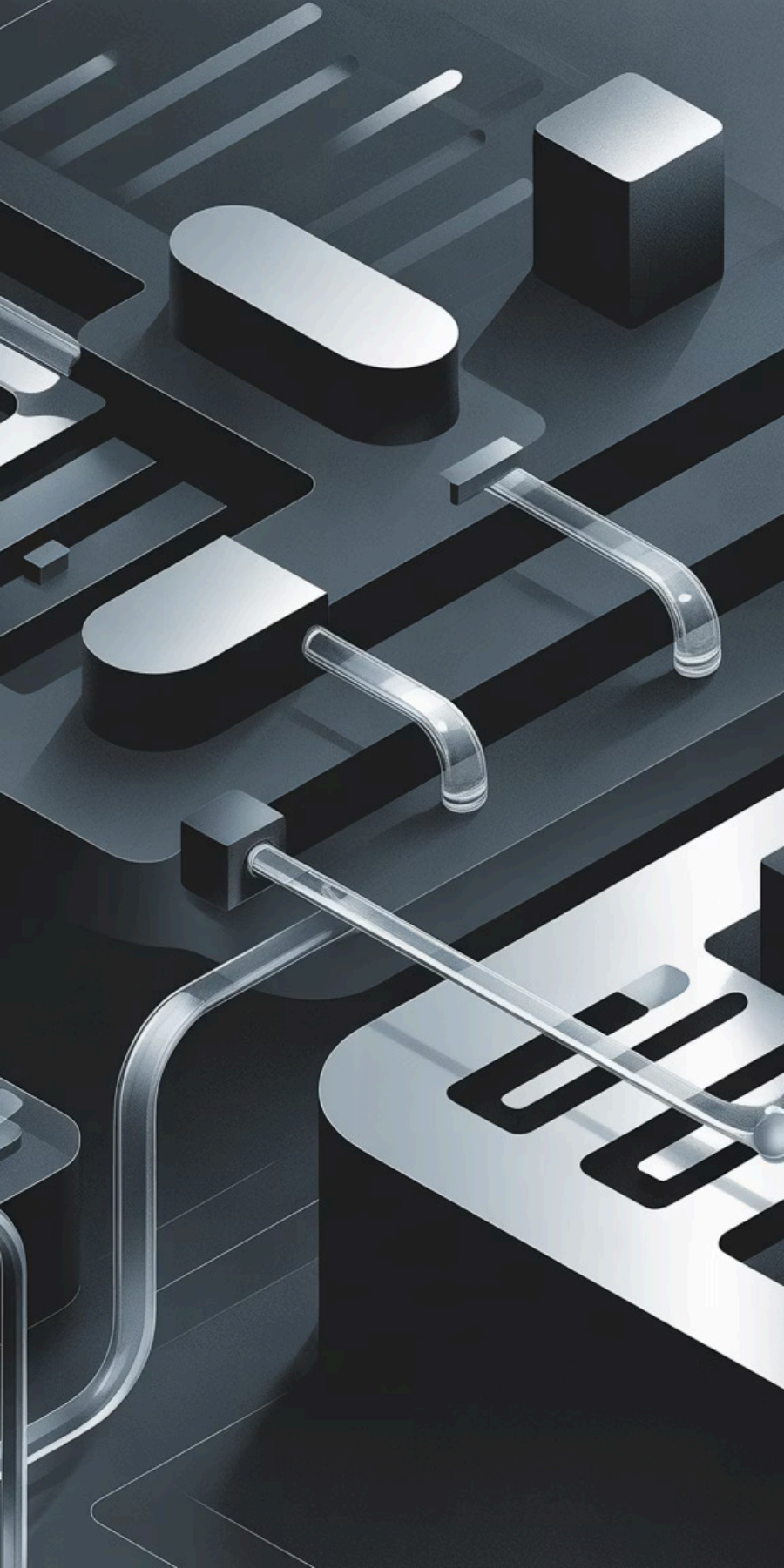
- API costs per request
- Latency and response time
- Context window limitations
- Processing throughput



Sweet spot for number of examples in most cases

Exemplars are input tokens — and input tokens cost money and latency on every single request. Trimming redundant words, tightening exemplar phrasing, and cutting the example count to the minimum that holds accuracy can materially reduce prompt size. *Actual savings vary widely by task* — measure yours on a held-out set rather than assuming a fixed percentage.

Optimize your examples by removing unnecessary words, using concise language, and avoiding redundancy while maintaining clarity.



Exemplar Selection Strategy

1

Coverage-Based Selection

Choose examples that represent different categories, formats, or edge cases in your data distribution.

2

Diversity Maximization

Select examples that are maximally different from each other to provide broad pattern recognition.

3

Error-Driven Refinement

When the model fails on specific inputs, add similar examples to your exemplar set.

4

Similarity-Based Dynamic Selection

Use embeddings to select examples most similar to each new input for maximum relevance.

Dynamic Few-Shot: Semantic Similarity Selection

Instead of using the same examples for every request, dynamically select the most relevant examples based on semantic similarity to the input. This approach significantly improves accuracy for diverse datasets.

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

example_texts = [ex['input'] for ex in example_pool]
example_embeddings = model.encode(example_texts)

def select_examples(new_input, k=3):
    new_embedding = model.encode([new_input])
    # cosine similarity, stated explicitly (portable across embedding models)
    similarities = model.similarity(new_embedding, example_embeddings)[0].numpy()
    top_indices = np.argsort(similarities)[-k:][::-1] # most-similar first
    return [example_pool[i] for i in top_indices]
```

Formatting Guidelines for Production Prompts



Consistent Structure

Use identical formatting across all examples. Same delimiters, same order, same spacing.



Parseable Output Formats

Prefer JSON, XML, or other structured formats that can be reliably parsed programmatically.



Explicit Field Labels

Name all output fields clearly. Don't rely on position or implicit structure.



Whitespace Consistency

Use consistent indentation and spacing. Helps both model and human readability.

JSON Output Format: Best Practices

✓ Good Example

```
{
  "entities": [
    {
      "text": "GPT-4",
      "type": "MODEL",
      "confidence": 0.95
    }
  ],
  "sentiment": "positive",
  "summary": "..."
}
```

Clear schema, consistent types, explicit null handling.

✗ Bad Example

```
{
  entities: GPT-4 (model),
  sentiment: good,
  summary: ...
}
```

Inconsistent quoting, ambiguous types, unreliable parsing.

Always instruct the model to output valid JSON and validate the response programmatically. Use Pydantic models or JSON Schema for validation.



Handling Edge Cases in Exemplars

Your exemplar set must explicitly demonstrate how to handle edge cases, or the model will make unpredictable decisions when encountering them.

- **Empty or Null Values**

Show how to represent missing data: empty lists, null fields, or explicit "N/A" markers.

- **Ambiguous Inputs**

Demonstrate consistent behavior for inputs that could map to multiple categories.

- **Out-of-Distribution Data**

Include examples that don't fit standard patterns and show how to handle them gracefully.

- **Malformed or Noisy Input**

Show how to extract information from imperfect data or return error indicators.

Example: Handling Missing Data

Extract paper metadata **from** citations. **Return** JSON **with** title, authors, **year**, venue.

Example 1:

Input: "Smith, J. (2023). Deep Learning Advances. In Proc. NeurIPS."

Output: {

"title": "Deep Learning Advances",

"authors": ["Smith, J."],

"year": 2023,

"venue": "NeurIPS"

}

Example 2:

Input: "Attention Mechanisms. Conference on AI, 2022."

Output: {

"title": "Attention Mechanisms",

"authors": [],

"year": 2022,

"venue": "Conference on AI"

}

Example 3:

Input: "An incomplete citation."

Output: {

"title": "An incomplete citation",

"authors": [],

"year": null,

"venue": null

}

Notice how the examples show progressively more missing fields, establishing the pattern for null handling.

Testing Your Few-Shot Prompts

01

Create a Test Set

Set aside 20-50 labeled examples that weren't used in your exemplar set.

02

Run Batch Evaluation

Process your test set and compare outputs to ground truth labels.

03

Calculate Metrics

Measure accuracy, F1 score, or task-specific metrics. Track performance across categories.

04

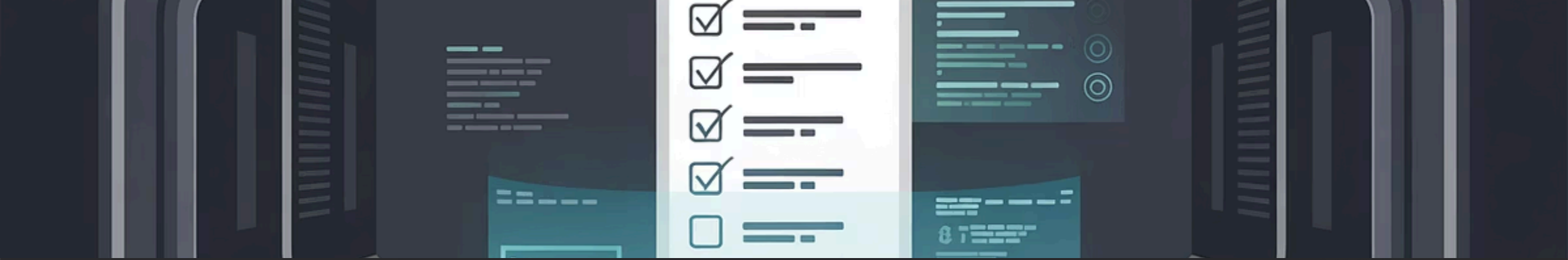
Analyze Failures

Identify patterns in errors. Do certain input types consistently fail?

05

Iterate on Examples

Add examples that address failure modes. Remove redundant examples.



Validation Pipeline for LLM Outputs

Never trust LLM outputs blindly. Build a robust validation pipeline that catches errors before they propagate downstream.

Schema Validation

Verify output structure matches expected format. Use JSON Schema or Pydantic for strict validation.

```
from pydantic import BaseModel

class EntityExtraction(BaseModel):
    models: list[str]
    datasets: list[str]
    metrics: list[str]

# Pydantic v2: parse + validate JSON in one call
result =
EntityExtraction.model_validate_json(llm_output)
```

Content Validation

Check that extracted values are reasonable. Implement domain-specific rules.

```
from datetime import datetime

def validate_year(year):
    # ceiling tracks the current year (tolerate next-year "in press")
    return 1900 <= year <= datetime.now().year + 1

def validate_metric(metric):
    allowed = ["accuracy", "f1", "precision", "recall"] # all lowercase
    return metric.lower() in allowed
```

Common Few-Shot Prompting Mistakes

Inconsistent Formatting

Using different delimiters or structures across examples confuses the model and leads to unpredictable outputs.

Too Many Examples

Including more than 10 examples wastes tokens and provides diminishing returns. Focus on quality over quantity.

Unrepresentative Examples

Selecting only simple or edge cases fails to establish the full pattern. Balance is crucial.

Ambiguous Instructions

Assuming the model will infer requirements from examples alone.
Always include clear task descriptions.

No Validation Layer

Trusting outputs without verification leads to silent failures in production pipelines.

Advanced Pattern: Chain-of-Thought in Few-Shot

For complex reasoning tasks, combine few-shot prompting with chain-of-thought reasoning. Show the model not just the answer, but the reasoning steps.

Classify research papers and explain the classification.

Example:

Title: "BERT: Pre-training of Deep Bidirectional Transformers"

Abstract: "We introduce BERT, which uses masked language modeling..."

Reasoning:

1. The title mentions "Transformers" - a core NLP architecture
2. Abstract describes "masked language modeling" - an NLP technique
3. BERT is a well-known language model
4. No mention of vision, robotics, or other domains

Classification: NLP

Confidence: High

This approach is especially valuable for classification tasks where understanding the decision process matters.



Production Considerations

Monitoring and Logging

- Log all inputs and outputs for debugging
- Track validation failure rates
- Monitor latency and token usage
- Set up alerts for anomalous patterns

Prompt Version Control

- Store prompts in version control (Git)
- Tag releases with performance metrics
- A/B test prompt variations in production
- Maintain rollback capability for bad prompts

Treat your prompts as critical infrastructure. Changes to exemplars can significantly impact system behavior, so apply the same rigor you use for code changes.

Prompt Engineering Workflow



When to Move Beyond Few-Shot Prompting

You Need Fine-Tuning If...

You have thousands of labeled examples, need maximum accuracy, or require specialized domain knowledge that few examples can't convey.

1

2

You Need RAG If...

Your task requires access to a large knowledge base, examples change frequently, or you need to incorporate up-to-date information.

You Need Specialized Models If...

Task-specific models (like NER or classification models) consistently outperform LLMs, or latency/cost constraints are critical.

3

Few-shot prompting excels at rapid prototyping, moderate-complexity tasks, and situations requiring flexibility. Know when it's the right tool.

Key Takeaways

Consistency is Everything

Format all exemplars identically. Use clear delimiters. Maintain structural consistency across examples.

Quality Over Quantity

3-5 well-crafted examples typically outperform 10 mediocre ones. Focus on representative coverage.

Always Validate

Never trust LLM outputs blindly. Build robust validation pipelines with schema and content checks.

Iterate Systematically

Test on held-out data. Analyze failures. Refine examples based on errors. Version control everything.



Thanks